
40 System Design / Architecture

Questions for LogIQ Debug Agent

1. How would you describe the high-level architecture of LogIQ Debug Agent?

LogIQ Debug Agent follows a layered architecture with a React frontend, a FastAPI backend, Neo4j for graph-based RCA reasoning, PostgreSQL for relational data like users and jobs, and an LLM layer for explanations. The backend orchestrates the debugging workflow and acts as the central control layer. The graph database handles relationship-heavy debugging data, while the relational database stores transactional and user-focused data. This separation makes the system easier to scale and maintain. It also keeps deterministic logic separate from AI-generated outputs.

2. Why did you choose a layered architecture instead of a monolithic script-based design?

A layered architecture improves maintainability, testability, and separation of concerns. The frontend handles presentation, the backend handles orchestration and business logic, and the databases handle persistence according to their strengths. A script-based design might work for experiments, but it becomes difficult to scale or extend. Since this project includes jobs, users, RCA computation, and AI explanations, a layered design makes each concern easier to evolve. It also makes the system more production-friendly.

3. Why did you choose FastAPI as the backend layer?

FastAPI is a strong fit because it is lightweight, high-performance, and works very well for API-first architectures. It supports asynchronous endpoints, strong schema validation with Pydantic, and clean route/service separation. In this system, the backend needs to manage jobs, trigger RCA, call databases, and integrate with the LLM layer. FastAPI makes those

integrations straightforward while keeping the codebase organized. It also provides automatic API docs, which helps development and testing.

4. Why did you choose Neo4j for RCA instead of keeping everything in PostgreSQL?

The RCA problem is relationship-heavy, which makes a graph database a better fit than a purely relational model. Debugging requires traversing entities like anomaly → stack frame → file → commit → deployment, and graph traversal is much more natural for that than multiple relational joins. Neo4j makes multi-hop reasoning easier to model and query. PostgreSQL is still useful, but more for structured transactional data. Using Neo4j improves expressiveness and aligns with the debugging domain.

5. Why did you still keep PostgreSQL in the architecture?

PostgreSQL is ideal for structured, transactional data such as users, roles, and job metadata. These entities require constraints, uniqueness, and predictable CRUD behavior, which relational databases handle very well. It would be inefficient to force all such data into the graph model. By keeping PostgreSQL for relational data and Neo4j for relationship-centric reasoning, the system uses each database where it fits best. This leads to a cleaner and more intentional architecture.

6. How do the frontend and backend communicate in your system?

The frontend communicates with the backend through HTTP APIs exposed by FastAPI. These APIs return structured JSON for jobs, anomalies, RCA results, user data, and explanations. The frontend is responsible for rendering that information into dashboards, investigation pages, and evidence panels. This API-driven separation allows the frontend and backend to evolve independently. It also makes testing and future integration easier.

7. How would you explain the core data flow in this system?

The system starts when a user creates or triggers a debugging investigation. That request goes to the backend, which creates a job and starts the RCA pipeline. The pipeline reads logs and stack traces, computes root cause candidates using graph-based reasoning, gathers evidence, and then passes the results to the LLM for explanation. Finally, the frontend fetches and renders the structured result. This data flow keeps deterministic analysis upstream and AI explanation downstream.

8. Why is deterministic RCA the source of truth in your architecture?

Deterministic RCA is the source of truth because debugging requires reliability, repeatability, and explainability. If the system used an LLM to decide root causes, results could vary and hallucinations could introduce incorrect conclusions. By basing RCA on structured signals like stack frames, graph relationships, and commit recency, the output remains factual. The LLM is only used after the fact to explain the already-computed result. This is a safer and more production-ready design pattern.

9. How does the RCA pipeline fit into the overall architecture?

The RCA pipeline is the core processing engine of the system. It sits inside the backend orchestration layer and runs a sequence of steps such as triage, RCA, evidence collection, memory lookup, explanation, remediation, and reporting. Each stage transforms the data and passes structured output to the next. This modular pipeline keeps responsibilities separated and makes each stage easier to improve independently. Architecturally, it acts like a workflow engine inside the backend.

10. Why did you break the system into pipeline stages instead of one large RCA function?

Breaking the system into stages improves modularity, observability, and extensibility. A single large function would be difficult to debug, test, and evolve as the system grows. By separating triage, RCA, evidence, explanation, and remediation, each step can be independently improved or replaced. It also aligns with an agent-style architecture, where each stage performs a focused responsibility. This is much easier to scale conceptually and technically.

11. How would you describe the job system from a system design perspective?

The job system acts as a control mechanism for long-running debugging tasks. Instead of blocking the user request while RCA runs, the backend creates a job record and processes it asynchronously. This allows the frontend to track state like queued, running, completed, or failed. It improves responsiveness and makes the system more scalable. It also provides a clear audit trail for investigations.

12. Why is asynchronous processing important in this architecture?

Asynchronous processing is important because root cause analysis and explanation generation can take longer than a normal API request. If everything ran synchronously, users would face slow responses and the backend would be less scalable. By processing RCA jobs asynchronously, the system can acknowledge requests quickly and continue work in the background. This improves both throughput and user experience. It also makes retries and monitoring easier.

13. Where would Redis fit in this architecture if you added it?

Redis would fit well as a caching and infrastructure support layer between the backend and heavier computations. It could cache RCA results, LLM outputs, and repeated queries to reduce latency. It could also support rate limiting, distributed locks, and job queue coordination. This would improve performance and production readiness. Redis would not replace the core databases, but it would make the architecture more efficient.

14. How would you architect caching for this system?

Caching should be applied to expensive and frequently reused outputs such as RCA results, explanation summaries, and maybe repeated graph lookups. A cache key could be based on the anomaly ID, job ID, or RCA input fingerprint. The backend would first check the cache

before recomputing. Cache invalidation would happen when new debugging data changes the result. This reduces repeated work and improves API responsiveness.

15. How would you scale the backend if usage increased significantly?

The backend can be scaled horizontally by deploying multiple FastAPI instances behind a load balancer. Since the APIs can be kept stateless, horizontal scaling is straightforward. Long-running tasks should be moved to background workers or a queue system rather than being tied to web instances. Redis and caching would reduce repeated computation, and database queries should be optimized. This architecture supports scaling without major redesign.

16. How would you scale Neo4j usage as graph data grows?

Scaling Neo4j begins with better graph modeling, indexing, and query optimization. Only essential entities and relationships should be stored to avoid graph bloat. High-cardinality traversals should be constrained with filters and bounded depth. For larger deployments, workload separation and Neo4j enterprise features could help. The main goal is to preserve traversal efficiency while the graph grows over time.

17. How would you scale the RCA pipeline itself?

The RCA pipeline can be scaled by decoupling it from the request-response layer and running it in worker processes. Different jobs can be processed in parallel if they are independent. Individual pipeline stages can also be optimized or parallelized, especially evidence collection or retrieval-heavy steps. Shared services like caching and rate limiting would reduce repeated work. This makes the pipeline more resilient under load.

18. Why is the frontend a separate application instead of server-rendered pages?

A separate React frontend provides better flexibility for building an interactive investigation experience. The UI includes dashboards, confidence scores, evidence panels, job tracking, and filtering, which benefit from client-side interactivity. It also cleanly separates presentation from backend logic. This architecture allows independent deployment and iteration of the frontend. For a developer-facing product, that flexibility is valuable.

19. How do you handle boundaries between deterministic logic and AI logic?

The system enforces a strict architectural boundary: deterministic logic computes the RCA, while AI logic interprets it. This ensures the LLM never becomes the authoritative source for root cause decisions. The output of deterministic analysis is structured and passed into the LLM only after the result is finalized. This keeps the system safe, predictable, and explainable. It is one of the most important architectural principles in the project.

20. How would you explain the AI layer in this architecture?

The AI layer is assistive rather than authoritative. It consumes structured RCA outputs and generates natural-language explanations, remediation steps, and summaries. This layer is isolated from the core decision-making logic so failures or hallucinations do not affect correctness. It adds usability for engineers and stakeholders. Architecturally, it is a downstream enrichment layer.

21. Where does GraphRAG fit into your system design?

GraphRAG fits into the retrieval and reasoning portion of the architecture. Instead of retrieving context using only vector similarity, it uses graph traversal across connected entities like stack frames, files, and commits. This makes retrieval more structural and explainable. In this system, GraphRAG is useful because debugging depends heavily on relationships. It strengthens the connection between graph storage and AI explanation.

22. How is GraphRAG different architecturally from standard vector RAG?

Standard vector RAG usually adds a vector database and retrieves semantically similar chunks of text. GraphRAG instead uses graph relationships as the primary retrieval mechanism. Architecturally, that means the retrieval layer depends more on graph structure than embedding similarity. In this system, that is valuable because debugging is not just a text similarity problem. It is a connected reasoning problem involving code and deployment relationships.

23. Would you add a vector database to this architecture? Why or why not?

Yes, but as a complement, not a replacement. A vector database would be useful for retrieving similar past incidents, log patterns, or documentation based on semantic similarity. However, it would not replace the graph because the graph handles structural relationships much better. So architecturally, a vector DB would sit alongside Neo4j as another retrieval mechanism. A hybrid approach would likely be the strongest design.

24. How do you keep the architecture flexible for future features?

The architecture stays flexible by separating responsibilities into layers and pipeline stages. New features like distributed tracing, authentication, advanced retrieval, or auto-remediation can be added without rewriting the core system. The backend acts as an orchestrator, which makes it easier to plug in new services or storage layers. The graph model can also be extended with additional node types. This modularity is what makes the system evolvable.

25. How would you add authentication to the architecture?

Authentication would be added at the backend API layer, likely using JWT or OAuth-based flows. The frontend would authenticate users and include tokens with requests. The backend would validate those tokens before allowing access to protected endpoints. User identity and

role information would continue to live in PostgreSQL or an auth provider. This would convert the current UI-only role model into real secured access control.

26. How would you add authorization properly?

Authorization should be enforced on the backend, not just in the UI. Each route or service layer should verify whether the authenticated user has permission for the requested action based on role or policy. This prevents users from bypassing the UI and directly calling protected endpoints. Role-based or policy-based checks can be added as middleware or dependencies. This is essential for production security.

27. How would you make the architecture more production-ready?

To make it production-ready, I would add authentication, backend authorization, rate limiting, caching, and proper observability. I would also move long-running RCA jobs to a dedicated worker queue and improve error handling across the pipeline. Monitoring, tracing, and structured logging would be added to understand system behavior in production. CI/CD and containerized deployment would be standardized. These improvements turn the architecture from strong prototype to deployable platform.

28. What are the main tradeoffs in your current architecture?

The main tradeoff is complexity versus capability. Using both Neo4j and PostgreSQL gives better modeling but increases operational overhead. Keeping RCA deterministic improves reliability but limits flexibility compared to fully AI-driven systems. A separate frontend and backend improve modularity but require API contract management. These tradeoffs were worth it because they align with correctness and scalability.

29. Why didn't you build this as a purely LLM-native system?

A purely LLM-native system would be easier to prototype but much harder to trust in production debugging. Root cause analysis requires factual reasoning and stable behavior, which LLMs cannot guarantee on their own. This project needed a system that engineers could rely on. That is why the architecture centers deterministic computation and uses AI only for explanation. It is a more responsible design for a debugging product.

30. Why didn't you keep everything as a single database system?

A single database would simplify operations, but it would weaken the model fit for at least one part of the system. Debugging relationships are naturally graph-shaped, while users and jobs are naturally relational. Combining both in one store would lead to awkward queries or less expressive modeling. By using two specialized databases, the architecture becomes clearer and more aligned with query patterns. The tradeoff is higher operational complexity.

31. How would you describe the knowledge graph schema at a high level?

The graph schema models important debugging entities as nodes and their relationships as edges. Core node types include anomaly, stack frame, file, commit, and deployment. Key relationships connect frames to files, commits to files, and RCA results back to files. This schema is designed to support graph traversal for root cause reasoning. It is intentionally centered around how engineers actually investigate issues.

32. How do you keep graph writes safe and consistent?

Graph writes should be done transactionally and only after key validations succeed. For example, evidence should be collected before writing final RCA attribution edges so the graph does not contain unsupported conclusions. Updates should also be isolated per anomaly or job to avoid cross-contamination. In production, locking or idempotent write patterns may be needed. The goal is to preserve graph integrity even during failures.

33. How would you handle repeated or duplicate RCA runs architecturally?

The architecture should prevent duplicate RCA runs by introducing idempotency and job checks. Before starting a new run, the system can verify whether there is already an active or completed result for the same anomaly. If so, it can reuse or reference the existing result unless explicitly overridden. This avoids wasted computation and inconsistent outputs. Architecturally, this belongs in the backend orchestration and job management layers.

34. How would you support historical debugging and recurring issue analysis?

Historical debugging can be supported by preserving past jobs, RCA results, and graph relationships rather than overwriting them. The graph can then be queried for patterns such as frequently implicated files or repeating anomaly types. A separate insights layer or reporting service can summarize these recurring signals. This turns the system from a reactive debugger into a learning platform. The architecture already supports this direction well.

35. How would you integrate distributed tracing into this architecture?

Distributed tracing would become another signal source alongside logs and stack traces. Trace spans could be ingested into the backend and either linked in the graph or referenced as supporting evidence. This would improve RCA by showing request flow across services. Architecturally, tracing would strengthen both the triage and evidence stages. It would also improve observability of the platform itself.

36. How would you design observability for the platform itself?

The platform should have structured logs, metrics, and traces for its own internal behavior. Metrics should track job volume, RCA latency, failure rates, cache hit rates, and LLM response times. Logs should include job IDs and anomaly IDs so events can be correlated. Tracing

should show how a debugging request moves through the backend and downstream services. This makes the platform easier to operate and debug.

37. How would you handle failures in the LLM layer without breaking the system?

The architecture should treat the LLM as optional enrichment rather than a required dependency. If the LLM fails, the deterministic RCA result should still be returned to the user. The UI can show the core result and indicate that explanation generation was unavailable. This fallback behavior ensures that the most important functionality never depends on the least deterministic component. It is a fail-safe AI design.

38. How would you describe the system from a microservices vs modular monolith perspective?

Right now, the architecture is closer to a modular monolith with strong internal separation. The frontend is separate, but the backend likely contains multiple logical modules rather than fully independent services. That is a good design at this stage because it keeps deployment and coordination simpler. As load or complexity grows, some modules such as job workers or AI services could be broken out. This is a pragmatic way to evolve architecture over time.

39. If an interviewer asks “what would you redesign next?”, what would you say?

The next redesign would focus on production hardening rather than changing the core concept. I would add authentication, backend authorization, Redis-based caching, and a dedicated background worker system. I would also improve observability and possibly add hybrid retrieval with a vector database for similar-incident lookup. The core architecture is strong, but these changes would make it more scalable and deployable. That shows a practical roadmap rather than unnecessary redesign.

40. What is the strongest architectural decision in your project?

The strongest architectural decision is separating deterministic root cause analysis from AI-generated explanation. That single decision makes the system trustworthy while still gaining the benefits of LLMs. It also naturally leads to a clean architecture where graph reasoning, orchestration, and AI enrichment are distinct layers. This improves correctness, maintainability, and interview defensibility. It is the design choice that gives the whole system credibility.

20 Follow-Up Grilling Questions for LogIQ Debug Agent

1. Why do you need Neo4j at all? Couldn't you do this with PostgreSQL joins?

Technically, some parts could be modeled in PostgreSQL, but the RCA problem is fundamentally relationship-heavy and multi-hop. Traversing anomaly → stack frame → file → commit → deployment repeatedly is much more natural and efficient in a graph model than in complex relational joins. Neo4j also makes the reasoning layer more expressive and easier to evolve as I add new entity types. PostgreSQL would work for structured metadata, but it would make the RCA traversal logic harder to maintain. So the decision was based on query patterns, not just storage preference.

2. Why not use only an LLM for root cause analysis and skip the deterministic engine?

Because RCA is a correctness-critical decision, not just a summarization task. An LLM can sound confident while being wrong, and in debugging that creates real operational risk. The deterministic engine gives me repeatable, auditable results based on stack traces, graph relationships, and commit signals. Then the LLM adds value by interpreting those results in a human-friendly way. That separation lets me benefit from AI without allowing it to compromise reliability.

3. If your deterministic RCA is wrong, then isn't the whole system wrong anyway?

That's a fair challenge, and the answer is that deterministic does not mean perfect — it means inspectable and improvable. If the RCA is wrong, I can trace exactly which scoring signals or graph paths led to that result and refine the logic. With an LLM-only system, it is much harder to debug why a conclusion was reached. So deterministic RCA gives me a controllable failure mode. It makes the system easier to validate, tune, and improve over time.

4. Your architecture uses both Neo4j and PostgreSQL. Isn't that overengineering?

It would be overengineering if both databases were storing similar data or if the problem were simple. In this case, the graph and relational parts solve genuinely different needs: graph traversal for RCA and structured transactional storage for users and jobs. Forcing everything into one database would simplify operations but reduce clarity and query fit. I made the tradeoff consciously because the domain benefits from specialized storage. The key is keeping the boundary between the two databases very clean.

5. What happens if Neo4j becomes slow as the graph grows large?

If Neo4j slows down, I would first look at graph design and query efficiency before assuming the database choice was wrong. That means pruning unnecessary nodes, adding indexes, limiting traversal depth, and tuning Cypher queries based on access patterns. I would also avoid treating the graph like a raw dump of all system events and instead keep it focused on RCA-relevant entities. If needed, I could introduce precomputed relationships or summary layers. Scaling graph usage is as much about disciplined modeling as infrastructure.

6. Why did you design the pipeline as sequential stages instead of parallelizing everything?

The sequential structure reflects dependency order: explanation depends on RCA, remediation depends on explanation context, and reporting depends on both. Parallelism is useful only when stages are independent, and not all stages here are. I chose clarity and correctness first, then left room to parallelize specific substeps like evidence collection later. That keeps the architecture understandable while still being extensible. Premature parallelism would have increased complexity without much benefit initially.

7. Why didn't you build this as microservices from the start?

Because at this stage, a modular monolith is a better tradeoff than early microservices. The system has multiple logical components, but splitting them into separate deployable services too early would add network, deployment, and observability complexity. I kept strong internal modularity so that extraction into services remains possible later. That approach preserves development speed while keeping the design clean. I prefer evolving into microservices only when team size, scaling, or ownership boundaries justify it.

8. If the LLM layer goes down, does your system still work?

Yes, by design the system should still provide its most important value even if the LLM is unavailable. The deterministic RCA result remains available because that logic is independent of the AI layer. In that case, the system can still show ranked root cause candidates and evidence, but without generated explanations or remediation text. This is an intentional fail-safe design choice. It keeps the core functionality resilient and prevents AI from becoming a single point of failure.

9. Why not let the LLM rerank the RCA candidates after the deterministic engine?

Because that would blur the trust boundary I intentionally created. Once the LLM starts reranking, it is influencing the authoritative result, which reintroduces non-determinism and explainability problems. I want the RCA score to come only from explicit, inspectable signals. If I ever wanted smarter ranking, I would rather improve the deterministic scoring model or add

well-validated learned features outside the generative layer. That keeps the final RCA decision grounded and defensible.

10. How do you know your graph schema is the right one and not just convenient?

The schema is driven by the debugging workflow, not by abstract graph modeling. Engineers investigate issues by moving across anomalies, traces, files, commits, and deployments, so those became the core entities. I chose nodes and relationships based on the reasoning paths needed for RCA, not just what data was available. A good schema should make important queries natural and unnecessary ones difficult. I would validate it by checking whether new RCA scenarios can be expressed without awkward workarounds.

11. Why do you need a job system? Why not just run RCA directly in the request?

Because RCA and explanation generation can be long-running and variable in duration. Keeping them inside the request-response cycle would make the API slower, less resilient, and harder to scale. A job system lets me decouple user interaction from backend processing, which improves responsiveness and observability. It also gives me a place to track retries, failures, and status transitions. That makes the system operationally much cleaner.

12. If two users trigger the same investigation at once, how do you avoid duplicate work?

This is where idempotency and concurrency control matter. I would use a uniqueness check based on the anomaly or investigation fingerprint and either reuse the existing active job or prevent a duplicate from starting. If multiple workers are involved, I'd likely add a distributed lock or atomic reservation step. That avoids inconsistent writes and wasted computation. The design goal is that the same logical RCA should not be recomputed unnecessarily unless explicitly requested.

13. Why not store all logs directly in the graph too?

Because not every piece of data belongs in the graph just because it is related to debugging. Graph storage should be reserved for entities and relationships that support traversal-heavy reasoning. Raw logs can be too high-volume and noisy, which would bloat the graph and hurt performance. A better design is to parse and extract structured signals from logs, then only store RCA-relevant abstractions in the graph. That keeps the graph useful rather than overloaded.

14. Why would you ever need a vector database if you already have a graph?

The graph helps with structural reasoning, but it does not solve semantic similarity well. A vector database would be useful for finding similar past incidents, log patterns, or documentation based on meaning rather than direct relationships. That complements the graph instead of replacing it. In practice, the graph answers “what is connected,” while the vector layer answers “what is similar.” A hybrid retrieval architecture is stronger than either one alone.

15. How would you defend using GraphRAG here if the interviewer says it sounds like hype?

I would explain it in concrete terms rather than using the buzzword alone. In this system, retrieval needs to respect relationships like which file belongs to which stack frame and which commit last touched that file. That is not just semantic similarity — it is connected reasoning. GraphRAG is simply a way of saying that retrieval is graph-structured instead of purely embedding-based. So the value is not the label; it is the architecture of the retrieval process.

16. What is the weakest architectural part of your current system?

The weakest part is probably production hardening rather than the core reasoning design. Right now, authentication, backend authorization, rate limiting, caching, and full observability are either missing or incomplete. That means the architecture is strong conceptually but still prototype-level operationally. I’m aware of that gap, and I see it as the next phase rather than a flaw in the core design. Being honest about that usually reads much stronger than pretending it is already production-complete.

17. How would you redesign this if your system had to support 100x more data and users?

I would keep the core architecture but strengthen the operational layers around it. That means moving RCA to dedicated worker services, adding Redis for caching and queue coordination, optimizing graph storage, and introducing stronger observability. I would also separate frequently accessed metadata from heavy analytical paths and tighten API boundaries. If necessary, some components could be extracted into services. The redesign would focus on scaling execution and storage, not changing the core reasoning model.

18. Why is your architecture better than a traditional observability dashboard with alerts?

Traditional dashboards are good at surfacing signals, but they still leave most of the reasoning burden on the engineer. My architecture goes one step further by converting those signals into a structured reasoning system that proposes root causes and evidence. It reduces the manual correlation work engineers normally do across logs, traces, and code changes. In other words, the goal is not just observability, but guided diagnosis. That is the real architectural distinction.

19. If the interviewer says “this sounds like a lot of moving parts,” how do you respond?

I'd say that's true, but the moving parts correspond to real responsibilities that already exist in debugging workflows. The architecture may look broad, but each part has a narrow purpose: graph for RCA reasoning, relational DB for structured metadata, backend for orchestration, and LLM for explanation. The system would be riskier if I collapsed those concerns into fewer but blurrier components. I optimized for separation of responsibilities rather than artificial simplicity. That usually makes systems easier to evolve in the long run.

20. What architectural decision would you defend the hardest if challenged?

I would defend the separation between deterministic RCA and AI explanation the hardest. That decision drives trust, correctness, and maintainability across the entire system. It prevents the AI layer from becoming the source of truth and gives the platform a much stronger reliability story. It also makes failures easier to reason about and reduces the risk of hallucinated debugging advice driving core conclusions. That one boundary is what makes the architecture credible.

(Focus: AI + RAG + Graph + Backend + System Design)

SECTION A: AI / RAG / LLM (HIGH PRIORITY)

1. What is RAG and why is it useful in your system?

RAG (Retrieval-Augmented Generation) is a technique where relevant context is retrieved before generating a response using an LLM. It improves accuracy by grounding responses in real data instead of relying only on model knowledge. In my system, RAG can be used to retrieve past incidents, logs, or RCA results before generating explanations. This ensures that the LLM produces context-aware and accurate outputs. It also reduces hallucinations by constraining the generation with factual data. Overall, RAG improves reliability and usefulness of AI outputs.

2. What is the difference between RAG and fine-tuning?

RAG retrieves external data at runtime, while fine-tuning modifies the model weights permanently. RAG is more flexible because it allows updating knowledge without retraining the model. Fine-tuning is useful when domain-specific behavior is needed consistently. In my system, RAG is preferred because debugging data changes frequently. It allows dynamic retrieval of logs and past RCA results. This makes the system scalable and easier to maintain.

3. What is GraphRAG and how is it different from traditional RAG?

GraphRAG uses a knowledge graph for retrieval instead of vector similarity. Traditional RAG relies on embeddings and nearest neighbor search. GraphRAG leverages relationships between entities, such as files and commits. This allows multi-hop reasoning, which is important for debugging. In my system, GraphRAG helps trace issues across multiple components. It provides more structured and explainable results compared to vector-based retrieval.

4. When would you choose vector RAG over GraphRAG?

Vector RAG is better when dealing with unstructured text like documents or logs. It works well for semantic similarity search. GraphRAG is better when relationships between entities are important. In my system, vector RAG can be used for log similarity, while GraphRAG is used for RCA reasoning. The choice depends on the type of data and query. Often, a hybrid approach works best.

5. What is an embedding?

An embedding is a numerical representation of text that captures semantic meaning. It allows comparison of text based on similarity. Embeddings are used in vector databases for retrieval. In RAG systems, they help find relevant documents. They are generated using models like BERT or OpenAI embeddings. This enables efficient search over large datasets.

6. What is a vector database and why is it used?

A vector database stores embeddings and enables similarity search. It is optimized for nearest neighbor queries. Examples include FAISS and Chroma. In my system, it can be used to store past logs and incidents. This allows retrieval of similar issues for faster debugging. It complements the graph database.

7. What is reranking in RAG?

Reranking is the process of reordering retrieved results based on relevance. Initial retrieval may return approximate matches. A reranker improves accuracy by scoring results more precisely. It often uses cross-encoder models or LLMs. In my system, reranking can improve log retrieval quality. This ensures better context for LLM generation.

8. How do you prevent hallucination in LLM systems?

Hallucination is prevented by grounding the LLM with structured data. In my system, RCA is deterministic and not handled by the LLM. The LLM only explains verified results. Structured prompts and context also reduce hallucination. Additionally, limiting the scope of generation helps. This ensures reliable outputs.

9. Why did you use local LLM instead of API-based LLM?

Using Ollama allows running models locally, reducing latency and cost. It also avoids dependency on external APIs. This is important for debugging systems where data privacy matters. Local models provide better control over behavior. The tradeoff is slightly lower performance compared to large cloud models.

10. What is prompt engineering and why is it important?

Prompt engineering involves designing inputs to guide LLM outputs. It ensures that the model produces accurate and relevant responses. Structured prompts improve consistency. In my system, prompts are designed to explain RCA results clearly. This reduces ambiguity in outputs. It is critical for reliable AI behavior.

11. What is the context window in LLMs?

The context window is the maximum amount of text an LLM can process at once. It limits how much information can be provided in a prompt. In RAG systems, selecting relevant context is important due to this limitation. In my system, only essential RCA data is passed to the LLM. This ensures efficient usage of the context window. It also improves response quality.

12. How do you select relevant context for RAG?

Relevant context is selected using retrieval techniques like vector search or graph traversal. Filtering is applied based on relevance scores. In my system, graph relationships help identify

important data. This reduces noise in the input. It ensures that the LLM receives only useful information. This improves output accuracy.

13. What is chunking in RAG?

Chunking involves splitting large documents into smaller pieces. This makes retrieval more efficient. Each chunk is embedded and stored separately. In my system, logs can be chunked for better retrieval. Proper chunk size is important for performance. It balances context completeness and retrieval accuracy.

14. What is a hybrid RAG system?

A hybrid RAG system combines multiple retrieval methods, such as vector search and graph traversal. This improves retrieval accuracy. In my system, vector search can find similar logs, while graph traversal provides structured relationships. Combining both gives better results. It leverages strengths of both approaches.

15. What is latency in RAG systems and how do you reduce it?

Latency is the time taken to retrieve context and generate a response. It can be reduced by caching results and optimizing queries. Using local LLMs also reduces network delay. Efficient indexing improves retrieval speed. In my system, caching RCA results can reduce latency. This improves user experience.



SECTION B: BACKEND + FASTAPI

16. Why did you choose FastAPI?

FastAPI provides high performance and async support. It is easy to build APIs with minimal boilerplate. It also offers automatic documentation. In my system, it handles RCA pipeline and job management. It is well-suited for microservices architecture.

17. What is async processing in FastAPI?

Async processing allows handling multiple requests without blocking. It improves scalability and performance. In my system, RCA jobs can run asynchronously. This prevents API delays. It ensures better resource utilization.

18. What is a background job system?

A background job system processes long-running tasks outside the request-response cycle. It improves responsiveness of APIs. In my system, RCA execution is handled as a background job. This allows users to track job status. It improves system scalability.

19. How would you implement rate limiting?

Rate limiting can be implemented using middleware or external tools like Redis. It restricts the number of requests per user. In my system, it can protect LLM endpoints. This prevents abuse and overload. It ensures fair usage.

20. What is caching and where would you use it?

Caching stores frequently accessed data for faster retrieval. It reduces computation and improves performance. In my system, RCA results and LLM responses can be cached. Redis can be used for this purpose. This reduces repeated processing.

SECTION C: GRAPH + SYSTEM DESIGN

21. Why use a knowledge graph for debugging?

Debugging involves relationships between multiple entities. A knowledge graph models these relationships effectively. It allows efficient traversal and reasoning. In my system, it helps identify root causes. It improves accuracy and explainability.

22. How does graph traversal help in RCA?

Graph traversal allows moving across connected nodes. It helps trace issues from stack frames to files and commits. This enables multi-step reasoning. In my system, traversal identifies root cause candidates. It is more efficient than relational joins.

23. What is a multi-hop query?

A multi-hop query involves traversing multiple relationships in a graph. It is useful for complex reasoning. In debugging, issues often span multiple components. In my system, multi-hop queries connect anomalies to deployments. This improves RCA accuracy.

24. What is graph indexing?

Graph indexing improves query performance by organizing data efficiently. It allows faster traversal. In Neo4j, indexes can be created on node properties. This improves RCA performance. It is important for scaling.

25. What are the tradeoffs of using Neo4j?

Neo4j provides efficient relationship queries but can be harder to scale horizontally. It requires careful data modeling. It may not be ideal for large-scale distributed systems. However, it excels in relationship-heavy use cases. In my system, the benefits outweigh the tradeoffs.

26. How is your job system designed?

The job system tracks the lifecycle of debugging tasks such as RCA execution. Each job has states like queued, running, completed, and failed. When a user triggers a debug operation, a job entry is created in the database. The backend processes the job asynchronously to avoid blocking API responses. This design allows scalability and monitoring of long-running tasks. It also enables retries and error tracking.

27. How do you handle failures in the RCA pipeline?

Failures are handled by capturing errors at each stage of the pipeline. The job system records error messages and updates the job status to failed. Logs are stored for debugging pipeline issues. The system ensures partial results do not corrupt the graph. Retry mechanisms can be implemented for transient failures. This improves reliability and fault tolerance.

28. How would you scale your backend?

The backend can be scaled horizontally by running multiple instances behind a load balancer. Stateless APIs make scaling easier. Background jobs can be handled using a distributed queue system. Database scaling involves read replicas and sharding strategies. Caching reduces load on databases. Overall, scaling is achieved through modular architecture.

29. What is API idempotency and why is it important?

Idempotency means that repeated API calls produce the same result. It is important for reliability, especially in distributed systems. In job creation APIs, duplicate requests should not create multiple jobs. This can be handled using unique request IDs. Idempotency prevents inconsistent states. It improves system robustness.

30. What is database normalization and do you use it?

Normalization organizes data to reduce redundancy. It ensures consistency and integrity. In PostgreSQL, user data is normalized into structured tables. However, Neo4j uses a denormalized graph structure. The system uses both approaches where appropriate. This balances performance and consistency.

31. How do you ensure data consistency between Neo4j and PostgreSQL?

Consistency is maintained by separating concerns between the two databases. PostgreSQL handles user data, while Neo4j handles graph data. Transactions are used to ensure atomic updates within each database. Cross-database consistency is managed at the application level. Proper error handling prevents partial updates. This ensures reliable data flow.

32. What is eventual consistency and where could it apply?

Eventual consistency means data may not be immediately consistent but will become consistent over time. It is useful in distributed systems. In this system, background jobs updating the graph can use eventual consistency. This improves performance and scalability. It is acceptable when immediate consistency is not required.

33. What is connection pooling?

Connection pooling manages database connections efficiently. Instead of creating new connections for each request, it reuses existing ones. This improves performance and reduces overhead. In FastAPI, connection pooling is handled via SQLAlchemy. It ensures efficient database interactions. It is essential for high-traffic systems.

34. How do you handle large log files?

Large log files are processed using chunking and streaming techniques. This prevents memory overload. Logs are parsed into structured formats incrementally. Only relevant data is stored in the graph. This improves performance and scalability. It also reduces processing time.

35. What is pagination and where would you use it?

Pagination divides large datasets into smaller chunks. It improves performance and user experience. In this system, job lists and logs can be paginated. This prevents overloading the frontend. It also reduces API response size. Pagination is essential for scalability.

SECTION B: REDIS + CACHING

36. What is Redis and why would you use it?

Redis is an in-memory key-value store used for caching and fast data access. It provides low latency and high throughput. In this system, Redis can cache RCA results and LLM responses. This reduces repeated computations. It also improves API performance. Redis can also be used for rate limiting.

37. What types of caching can you implement?

Caching can be implemented at multiple levels, such as application-level, database-level, and API-level. In-memory caching like Redis is commonly used. Response caching can store API results. Query caching reduces database load. In this system, RCA and LLM outputs are ideal for caching. This improves performance significantly.

38. What is cache invalidation?

Cache invalidation ensures outdated data is removed from cache. It is important for maintaining data accuracy. Strategies include time-based expiration and manual invalidation. In this system, cache should be invalidated when new RCA results are generated. This ensures fresh data is served. It balances performance and accuracy.

39. How would you implement rate limiting using Redis?

Rate limiting can be implemented using counters stored in Redis. Each request increments a counter for a user. If the count exceeds a threshold, requests are blocked. Redis supports atomic operations for accuracy. Time-based expiration resets counters. This ensures fair usage and prevents abuse.

40. What is TTL in Redis?

TTL (Time-To-Live) defines how long data stays in cache. After expiration, the data is automatically removed. It helps manage memory efficiently. In this system, TTL can be set for cached RCA results. This ensures outdated data is not used. It simplifies cache management.



SECTION C: AGENTIC AI + PIPELINES

41. What is an agent in AI systems?

An agent is a system that performs tasks autonomously based on inputs and goals. It can make decisions and execute actions. In this system, each pipeline stage acts like an agent. For

example, the RCA agent computes root causes. The explanation agent generates insights. This modular design improves flexibility.

42. What is a multi-agent system?

A multi-agent system consists of multiple agents working together. Each agent handles a specific task. They communicate and coordinate to achieve a goal. In this system, triage, RCA, and explanation are separate agents. This improves scalability and maintainability. It also allows independent upgrades.

43. Why use an agent-based architecture?

Agent-based architecture improves modularity and flexibility. Each component can evolve independently. It allows parallel execution of tasks. It also simplifies debugging and testing. In this system, it aligns well with pipeline stages. It supports future extensions.

44. What is orchestration in agent systems?

Orchestration manages how agents interact and execute tasks. It defines the flow of execution. In this system, the pipeline orchestrates agents sequentially. It ensures correct order of operations. It also handles dependencies between agents. This ensures smooth execution.

45. What is the difference between agents and microservices?

Agents focus on decision-making and task execution. Microservices focus on delivering specific functionalities. Agents may use microservices internally. In this system, the pipeline acts like agents within a backend service. The distinction depends on abstraction level. Both improve modularity.

46. How do agents communicate in your system?

Agents communicate through structured data passed between pipeline stages. Each stage produces output consumed by the next stage. This ensures consistency. It avoids tight coupling. It also simplifies debugging. Communication is handled within the backend.

47. What is a planner in agent systems?

A planner decides which actions an agent should take. It determines execution flow dynamically. In advanced systems, planners use LLMs. In this system, the pipeline is predefined, so no dynamic planner is needed. However, future versions can include it. This would enable adaptive debugging.

48. What is tool usage in LLM agents?

Tool usage allows LLMs to call external functions. This extends their capabilities. In debugging, tools can fetch logs or query databases. In this system, tools can support explanation generation. This improves accuracy. It enables real-world integration.

49. What is memory in agent systems?

Memory stores past interactions and results. It helps agents learn from previous tasks. In this system, RCA results can be stored as memory. This helps identify recurring issues. It improves efficiency. It also enhances context for future runs.

50. What are limitations of agent systems?

Agent systems can become complex and hard to debug. Coordination between agents can introduce latency. They may require careful orchestration. Overuse of agents can lead to inefficiency. In this system, simplicity is maintained by using a structured pipeline. This avoids unnecessary complexity.

SECTION D: KNOWLEDGE GRAPH (DEEP)

51. How do you model your knowledge graph?

The graph consists of nodes like anomalies, stack frames, files, commits, and deployments. Relationships define how these nodes are connected. Each node contains relevant attributes. This structure supports efficient traversal. It enables RCA reasoning. Proper modeling is critical for performance.

52. What is a node vs relationship?

A node represents an entity, while a relationship connects two nodes. Relationships define how entities interact. In this system, nodes include files and commits. Relationships include IN_FILE and TOUCHED. This structure enables graph queries. It supports debugging logic.

53. What is graph traversal complexity?

Graph traversal complexity depends on the number of nodes and edges. Efficient indexing reduces traversal time. In Neo4j, traversal is optimized for relationships. This makes queries faster than relational joins. It is suitable for debugging use cases. Complexity is manageable with proper design.

54. How do you optimize graph queries?

Graph queries can be optimized using indexes and query tuning. Limiting traversal depth improves performance. Filtering nodes reduces search space. Proper data modeling is important. In this system, only relevant nodes are traversed. This ensures efficient RCA.

55. What is a Cypher query?

Cypher is the query language used in Neo4j. It allows pattern-based queries on graphs. It is intuitive and easy to read. In this system, Cypher is used for RCA traversal. It enables efficient data retrieval. It simplifies graph operations.



QUESTIONS 56–75

FRONTEND

56. Why did you choose React for the frontend?

I chose React because it is component-based and works well for building interactive dashboards and investigation workflows. The project has multiple reusable UI parts like RCA cards, confidence bars, evidence panels, and job tables, so React makes that easier to structure and maintain. It also has a strong ecosystem and integrates well with TypeScript, routing, and charting libraries. Since this is a developer-facing product, responsiveness and state-driven rendering are important, and React handles that very well. It also supports gradual scaling as the UI grows. Overall, it was a strong fit for a modern full-stack application.

57. Why did you use TypeScript in the frontend?

TypeScript helps catch errors early by enforcing type safety across components, props, API responses, and state. In a project like this, the frontend consumes structured backend data such as jobs, anomalies, RCA candidates, and user roles, so type safety reduces bugs significantly. It also improves code readability and maintainability, especially as the codebase grows. For team collaboration, TypeScript makes interfaces and expected data shapes explicit. It is especially useful when backend contracts evolve. Overall, it improves reliability and developer productivity.

58. Why did you use Vite instead of Create React App?

Vite provides a much faster development experience, especially for startup time and hot module replacement. That matters when iterating quickly on UI features like dashboards, charts, and detailed views. It also has a simpler and more modern build setup than Create React App. Since the project uses React, TypeScript, and Tailwind, Vite integrates cleanly with that stack. It reduces tooling overhead and improves local development speed. For a fast-moving product, that is a practical advantage.

59. How do you manage frontend state in this project?

Frontend state is managed based on the type of data being handled. Local UI state, like tab selection or modal visibility, can stay inside components, while shared data like jobs or RCA details can be managed at a higher level or through hooks. API-driven state is typically fetched from the backend and then rendered in structured views. For this kind of application, separating local state from server state is important to keep the UI predictable. As the project grows, this

can be improved further with tools like React Query for caching and synchronization. The goal is to keep the UI simple, responsive, and easy to reason about.

60. How does the frontend interact with the backend?

The frontend calls backend APIs exposed by FastAPI for operations like creating jobs, listing investigations, fetching anomalies, and showing RCA results. These calls are usually made using HTTP requests and return structured JSON responses. The frontend then maps that data into reusable UI components like cards, tables, and charts. This separation keeps the frontend focused on presentation and interaction while the backend handles business logic. It also makes the system easier to test and scale independently. Clear API contracts are important for keeping both layers aligned.

61. How do you handle role-based UI in the frontend?

Role-based UI is handled by reading the user's role from backend-provided user data and conditionally rendering features based on that role. For example, a viewer may only see read-only dashboards, while a developer or SRE may see debugging tools and investigation actions. This improves usability by reducing noise and showing relevant workflows to the right users. However, in the current version, this is UI-level control and not true backend authorization. That means it helps guide the experience but should not be treated as a security boundary. A production version would enforce roles on the backend as well.

62. How do you design the UI for technical users like developers or SREs?

For technical users, the UI should prioritize clarity, density of useful information, and quick navigation between related signals. Instead of overly visual or generic dashboards, the interface should show structured debugging artifacts like stack traces, confidence scores, file candidates, and evidence links. The goal is to reduce the time required to move from issue detection to understanding. Technical users also care about transparency, so the system should explain why a file was identified as a likely root cause. Good information hierarchy is critical here. The UI should feel like a debugging workspace, not just a reporting screen.

63. How do you visualize RCA results in the UI?

RCA results are best visualized using ranked candidate cards or tables with confidence scores, file names, and supporting evidence. This lets users quickly see the most likely root causes and

compare them. Additional sections can show related stack frames, commits, and explanation summaries. Confidence indicators help users understand prioritization without reading everything first. The UI should also allow drilling down into evidence so the result does not feel like a black box. That balance between summary and traceability is important.

64. How would you improve frontend performance in this application?

Frontend performance can be improved by reducing unnecessary re-renders, code-splitting large pages, and caching API results where appropriate. For repeated views like job details or RCA pages, keeping data cached avoids refetching every time the user navigates back. Lazy loading can help for heavier pages like insights or utilities. Memoization can also be used for expensive computed UI transformations. Efficient pagination and filtering reduce the amount of data rendered at once. Overall, good state design and API efficiency both contribute to frontend performance.

65. How do you handle API errors in the frontend?

API errors should be handled gracefully by showing meaningful feedback instead of failing silently. For example, if an RCA job fails, the user should see the failure state, error summary, and possibly a retry option. Error boundaries and request-level error handling improve resilience. Loading, success, empty, and failure states should all be designed explicitly. This is especially important in a debugging product, where users already come in during stressful situations. A clear error experience improves trust in the system.

FASTAPI / BACKEND DEEPER

66. How do you structure a FastAPI project for maintainability?

A maintainable FastAPI project should separate concerns into layers such as routes, services, schemas, models, and database utilities. Routes should handle HTTP input/output, while service layers should contain business logic like RCA execution or job orchestration. Pydantic schemas help validate request and response data. Database access should be isolated so it is easy to swap implementations or test components. This structure keeps the codebase modular and easier to scale. It also reduces tight coupling between API endpoints and internal logic.

67. Why is Pydantic useful in FastAPI?

Pydantic is useful because it validates and serializes request and response data automatically. That means the API can enforce expected structures for jobs, users, anomalies, and RCA results. It reduces manual validation code and improves safety when data moves across layers. It also helps generate clear API documentation. In this project, that is especially useful because the backend passes structured outputs to the frontend and the AI layer. Strong schema validation makes the system more reliable.

68. What is dependency injection in FastAPI?

Dependency injection in FastAPI is a way to provide shared components like database sessions, authentication logic, or configuration into endpoints without hardcoding them. It improves modularity and testability. For example, a route can depend on a database session provider instead of creating the session itself. That keeps endpoint logic cleaner and easier to reuse. In larger systems, dependency injection also makes mocking easier in tests. It is one of the reasons FastAPI scales well for structured backend development.

69. How do you design request and response models for APIs?

Request and response models should be explicit, stable, and focused only on the required data. Request models define what the client must send, while response models define exactly what the API returns. In this project, response models are important because the frontend depends on consistent job states, RCA candidates, evidence, and explanation data. Keeping models clean prevents leaking internal implementation details. It also makes versioning easier later. Good API contracts reduce confusion between frontend and backend teams.

70. What is the difference between synchronous and asynchronous endpoints?

Synchronous endpoints process requests in a blocking manner, while asynchronous endpoints allow non-blocking execution for I/O-heavy tasks. Async endpoints are useful when the application spends time waiting on databases, files, or external services. In this system, async patterns are helpful for job-related flows and integration-heavy operations. However, not every endpoint needs to be async; it should be used where it provides real benefit. Mixing both appropriately is usually the best approach. The goal is better throughput, not using async everywhere blindly.

DATABASE / DATA MODELING

71. Why use both Neo4j and PostgreSQL instead of one database?

The two databases serve different purposes and match different data shapes. PostgreSQL is good for structured relational data like users, roles, and job metadata, while Neo4j is better for highly connected debugging relationships. Trying to force both use cases into one database would make either the graph logic or the relational logic less natural. This hybrid design lets each database do what it does best. It also keeps the architecture clearer by separating business domains. The tradeoff is added operational complexity, but the modeling benefit is worth it.

72. How do you decide what data goes into Neo4j versus PostgreSQL?

Data goes into Neo4j when relationships are central to how the data will be queried, such as anomalies, stack frames, files, commits, and deployments. Data goes into PostgreSQL when it is more relational and transactional, such as user records and roles. The decision is based on query patterns, not just data type. If the value comes from traversing connections, graph storage is the better fit. If the value comes from structured CRUD operations and constraints, relational storage is usually better. This keeps the system efficient and maintainable.

73. What are the benefits of PostgreSQL for user management?

PostgreSQL is strong for user management because it supports structured schemas, constraints, unique keys, and transactional consistency. In this system, fields like email uniqueness and role assignment are naturally modeled in a relational database. It is also mature, reliable, and well supported with ORMs and migrations. User data usually needs consistency more than graph traversal. PostgreSQL handles that very well. It is a practical and production-friendly choice for this part of the system.

74. What are indexes and why are they important?

Indexes improve query performance by allowing the database to find records faster without scanning everything. They are especially important for frequently queried fields like email, job ID, or graph node identifiers. In PostgreSQL, indexes help speed up user lookups and job queries. In Neo4j, indexes help quickly find nodes before performing graph traversal. Without proper indexing, performance degrades as data grows. Good indexing is one of the most important optimization tools in production systems.

75. What are the tradeoffs of a hybrid data architecture?

A hybrid architecture gives better modeling flexibility because each database is used for the type of data it handles best. It improves performance and keeps graph reasoning separate from relational operations. However, it also increases system complexity because developers must manage two data stores, two query styles, and application-level coordination. Operational overhead such as backups, migrations, and observability also becomes more involved. Despite that, for this project, the separation is valuable because debugging relationships and user management are fundamentally different problems. The key is to keep the boundaries between the two databases clear.

QUESTIONS 76–100

REDIS (DEEP)

76. How would you use Redis in your system beyond caching?

Redis can be used for multiple purposes beyond caching, such as rate limiting, distributed locking, and job queues. In this system, Redis can act as a message broker for background jobs like RCA execution. It can also store session data or temporary computation results. Another use case is storing counters for monitoring or throttling. Because Redis is in-memory, it provides very low latency. This makes it ideal for real-time features in debugging workflows.

77. What is a distributed lock and why would you need it?

A distributed lock ensures that only one process performs a specific task at a time across multiple instances. This is useful in preventing duplicate RCA runs for the same anomaly. Redis can implement distributed locks using atomic operations. Without locks, race conditions could lead to inconsistent graph updates. This is especially important in horizontally scaled systems. It ensures data integrity.

78. What is a message queue and how does it help?

A message queue allows asynchronous communication between components. Tasks are placed in a queue and processed by workers independently. In this system, RCA jobs can be queued and processed by worker services. This decouples request handling from heavy computation. It improves scalability and fault tolerance. Redis or tools like Celery can be used for this.

79. What is pub/sub in Redis?

Pub/Sub is a messaging pattern where publishers send messages to channels and subscribers receive them. It enables real-time communication. In this system, it can be used to notify the frontend when a job is completed. This reduces the need for polling. It improves responsiveness of the UI. It is useful for real-time debugging updates.

80. What are Redis persistence options?

Redis supports persistence through RDB snapshots and AOF logs. RDB saves snapshots at intervals, while AOF logs every write operation. This ensures data durability despite being in-memory. In this system, persistence may be optional for cache but important for queues. Choosing the right persistence strategy depends on use case. It balances performance and reliability.

SECURITY

81. What are the current security limitations in your system?

Currently, the system lacks full authentication and backend-enforced authorization. Role-based access is implemented only at the UI level. This means users could potentially bypass restrictions by calling APIs directly. There is also no rate limiting yet. These are acceptable for a prototype but not for production. Addressing these is a key next step.

82. How would you implement authentication?

Authentication can be implemented using JWT-based tokens. Users would log in and receive a token that is sent with each request. The backend validates the token before processing requests. This ensures only authorized users access the system. Integration with OAuth providers is also possible. This improves security and scalability.

83. What is authorization and how is it different from authentication?

Authentication verifies who the user is, while authorization determines what the user can do. In this system, roles like developer or SRE define access levels. Authorization should be enforced at the backend, not just UI. This prevents unauthorized API access. It ensures proper security boundaries. Both are essential for production systems.

84. How would you secure your APIs?

APIs can be secured using authentication tokens, HTTPS, and input validation. Rate limiting prevents abuse. Proper error handling avoids exposing sensitive data. Logging and monitoring help detect suspicious activity. In this system, securing LLM endpoints is especially important. This ensures system integrity.

85. What is input validation and why is it important?

Input validation ensures that incoming data meets expected formats and constraints. It prevents malformed or malicious data from entering the system. In FastAPI, Pydantic models handle validation. This reduces security risks like injection attacks. It also improves system stability. Validation is a basic but critical practice.

CI/CD + DEVOPS

86. What is CI/CD and how is it used in your project?

CI/CD stands for Continuous Integration and Continuous Deployment. It automates building, testing, and deploying code. In this project, GitHub Actions can be used to deploy the frontend to Vercel. Backend deployments can also be automated. This reduces manual effort and errors. It ensures faster and reliable releases.

87. What is a CI pipeline?

A CI pipeline runs automated steps when code is pushed. It typically includes building, testing, and linting. This ensures code quality before merging. In this system, frontend and backend can have separate pipelines. This improves development workflow. It catches issues early.

88. What is a CD pipeline?

A CD pipeline automates deployment after successful builds. It pushes code to production or staging environments. This ensures continuous delivery of features. In this system, frontend deployment can be automated via Vercel. Backend deployment can use containers or cloud services. It reduces downtime.

89. What are environment variables and why are they important?

Environment variables store configuration values like database URLs and API keys. They keep sensitive data out of code. In this system, variables like `POSTGRES_URL` are critical. They allow different configurations for dev, staging, and production. This improves security and flexibility. It is a standard practice.

90. What is infrastructure as code (IaC)?

IaC defines infrastructure using code instead of manual setup. Tools like Terraform can be used. It ensures reproducibility and consistency. In this system, IaC can manage databases and services. It simplifies scaling and deployment. It is essential for modern DevOps.



DOCKER + DEPLOYMENT

91. Why use Docker in your system?

Docker containers package the application with its dependencies. This ensures consistent environments across development and production. In this system, backend, database, and

services can be containerized. It simplifies deployment and scaling. It also improves portability. This is essential for modern systems.

92. What is Docker Compose?

Docker Compose allows running multiple containers together. It defines services like backend, database, and Redis in one file. This simplifies local development. In this system, Compose can manage FastAPI, Neo4j, and PostgreSQL. It ensures everything runs consistently. It reduces setup complexity.

93. How would you deploy your backend?

The backend can be deployed using containerized services on cloud platforms. Options include AWS, Azure, or Render. A load balancer can distribute traffic across instances. Environment variables configure runtime settings. Monitoring tools track performance. This ensures reliability and scalability.

94. What is a load balancer?

A load balancer distributes incoming traffic across multiple servers. This prevents any single server from being overloaded. It improves availability and fault tolerance. In this system, it can distribute API requests. It enables horizontal scaling. It is critical for production systems.

95. What is horizontal vs vertical scaling?

Horizontal scaling adds more instances, while vertical scaling increases resources of a single instance. Horizontal scaling improves availability and fault tolerance. Vertical scaling is simpler but limited. In this system, horizontal scaling is preferred. It supports growth and resilience.



OBSERVABILITY + PRODUCTION

96. What is observability?

Observability is the ability to understand system behavior through logs, metrics, and traces. It helps detect and debug issues. In this system, observability is essential for monitoring RCA pipelines. Tools like Prometheus and Grafana can be used. It improves reliability. It supports production readiness.

97. What is logging and how is it used?

Logging records system events and errors. It helps track system behavior and debug issues. In this system, logs are central to RCA. Structured logging improves analysis. Logs should include timestamps and context. This makes debugging easier.

98. What is monitoring?

Monitoring tracks system health and performance metrics. It helps detect issues before they impact users. Metrics like response time and error rate are important. In this system, monitoring ensures pipeline stability. Alerts can notify engineers of issues. It improves reliability.

99. What is distributed tracing?

Distributed tracing tracks requests across multiple services. It helps identify bottlenecks and failures. In this system, it can trace RCA pipeline steps. It improves debugging of the system itself. Tools like OpenTelemetry can be used. It enhances observability.

100. What would you do to make this system production-ready?

To make the system production-ready, I would add authentication, backend authorization, and rate limiting. I would implement caching using Redis and introduce a proper job queue. Observability with logging, metrics, and tracing would be added. CI/CD pipelines and containerized deployment would be finalized. Finally, I would focus on scalability and security improvements. This would ensure reliability and robustness.

TOP 20 MUST-KNOW ANSWERS (LogIQ Debug Agent)

1. Explain your project in 60 seconds

LogIQ Debug Agent is an AI-powered debugging platform that automates root cause analysis for production issues. It combines deterministic RCA with a knowledge graph built using Neo4j and an LLM layer via Ollama. The system converts logs and stack traces into structured graph data, identifies root causes using scoring and traversal, and then generates explanations and remediation steps. The key design principle is that RCA is deterministic, while the LLM is only used for interpretation. This ensures both reliability and usability.

2. Walk me through the system architecture

The system has a layered architecture with a FastAPI backend, a Neo4j graph database for RCA reasoning, PostgreSQL for user management, and a React frontend. The backend orchestrates a pipeline: triage, RCA, evidence collection, explanation, and remediation. The graph database stores relationships between anomalies, stack frames, files, and commits. The LLM layer generates explanations after RCA is computed. This separation of concerns keeps the system modular and scalable.

3. How does your RCA system work?

The RCA system is deterministic and uses structured signals rather than AI predictions. It analyzes stack traces, traverses the graph to identify related files and commits, and applies scoring based on factors like frame depth and commit recency. Each candidate file is ranked with a confidence score. The top-ranked files are considered the root cause. This ensures repeatable and explainable results.

4. Why did you use a graph database?

Debugging is inherently relational, involving connections between stack frames, files, commits, and deployments. Using Neo4j allows efficient traversal of these relationships without complex

joins. It enables multi-hop reasoning, which is critical for RCA. Compared to relational databases, graph queries are more intuitive for this use case. The tradeoff is added complexity, but the performance and clarity benefits are significant.

5. Why not use LLM for RCA?

LLMs are probabilistic and can produce inconsistent or hallucinated outputs, which is risky for debugging. RCA requires deterministic and reliable results. In my system, RCA is computed using structured logic and graph traversal. The LLM is only used after RCA to explain findings and suggest remediation. This separation ensures correctness while still leveraging AI for usability.

6. What is GraphRAG and how do you use it?

GraphRAG is a retrieval approach that uses graph relationships instead of vector similarity. It allows retrieving context by traversing connections between entities. In my system, it helps trace issues across stack frames, files, and commits. This enables more structured and explainable reasoning compared to vector-based retrieval. It is especially useful for debugging workflows.

7. What is the role of the LLM in your system?

The LLM is used for generating explanations, summaries, and remediation steps. It takes structured RCA output and converts it into human-readable insights. This helps engineers quickly understand issues without deep log analysis. It improves usability but does not influence decision-making. This keeps the system reliable.

8. Why did you choose FastAPI?

I used FastAPI because it is lightweight, high-performance, and supports asynchronous processing. It is well-suited for building APIs that handle job orchestration and RCA pipelines. FastAPI also provides automatic documentation and integrates well with Pydantic for validation. This makes development faster and more structured.

9. Explain your pipeline (agent system)

The system follows a structured pipeline: triage → RCA → evidence → memory → explanation → remediation. Each stage acts like an agent responsible for a specific task. Data flows sequentially between stages, ensuring modularity. This design allows independent improvements in each step. It also aligns with agent-based architecture principles.

10. How do you handle long-running tasks?

Long-running tasks like RCA are handled asynchronously using a job system. When a job is triggered, it is processed in the background instead of blocking the API. The job status is tracked and updated as it progresses. This improves responsiveness and scalability. It also allows retries and failure handling.

11. What is RAG and where would you use it?

RAG (Retrieval-Augmented Generation) retrieves relevant data before generating responses. It improves accuracy by grounding LLM outputs in real data. In my system, RAG can be used to fetch similar past incidents or logs. This provides better context for explanations. It reduces hallucination and improves relevance.

12. Where would you use caching?

Caching can be used for RCA results and LLM responses, as these are expensive to compute and often reused. Using Redis can significantly reduce latency. It also reduces load on the backend and databases. Proper cache invalidation ensures data freshness. This improves overall system performance.

13. What are current limitations of your system?

The system currently lacks full authentication and backend-enforced authorization. Role-based access is implemented only at the UI level. There is no rate limiting or caching layer yet. Some modules still use mock data. These are areas I would improve to make it production-ready.

14. How would you scale this system?

The backend can be scaled horizontally using multiple instances behind a load balancer. Background jobs can be handled using distributed queues. The graph database can be optimized with indexing and query tuning. Caching reduces load on the system. Overall, the modular design supports scaling.

15. How do you prevent hallucination?

Hallucination is prevented by keeping RCA deterministic and separate from the LLM. The LLM only works on structured, verified data. Prompts are designed to constrain outputs. This ensures the model does not invent information. It improves trust in the system.

16. Why use both Neo4j and PostgreSQL?

The two databases serve different purposes. Neo4j handles relationship-heavy data for RCA, while PostgreSQL manages structured user data. This separation improves performance and clarity. Each database is used where it fits best. The tradeoff is added complexity.

17. What is an agent in your system?

An agent represents a functional unit in the pipeline, such as RCA computation or explanation generation. Each agent performs a specific task and passes results to the next stage. This modular design improves flexibility and maintainability. It also allows independent scaling of components. It aligns with agent-based system design.

18. How would you improve this system?

I would add authentication, backend authorization, and rate limiting. I would introduce caching using Redis and a proper job queue system. Observability with logging and metrics would be added. I would also integrate distributed tracing. These improvements would make the system production-ready.



19. What are tradeoffs in your design?

The main tradeoff is between flexibility and complexity. Using both graph and relational databases increases system complexity. Keeping RCA deterministic limits flexibility but ensures correctness. Using local LLMs reduces cost but may limit performance. These tradeoffs were chosen to prioritize reliability and clarity.



20. What makes your project unique?

This project stands out because it combines deterministic RCA with graph-based reasoning and AI explanations. Most systems rely heavily on LLMs, but this system ensures correctness first. The use of a knowledge graph for debugging is also relatively unique. It is a full-stack system with real-world applicability. It demonstrates both engineering and system design skills.